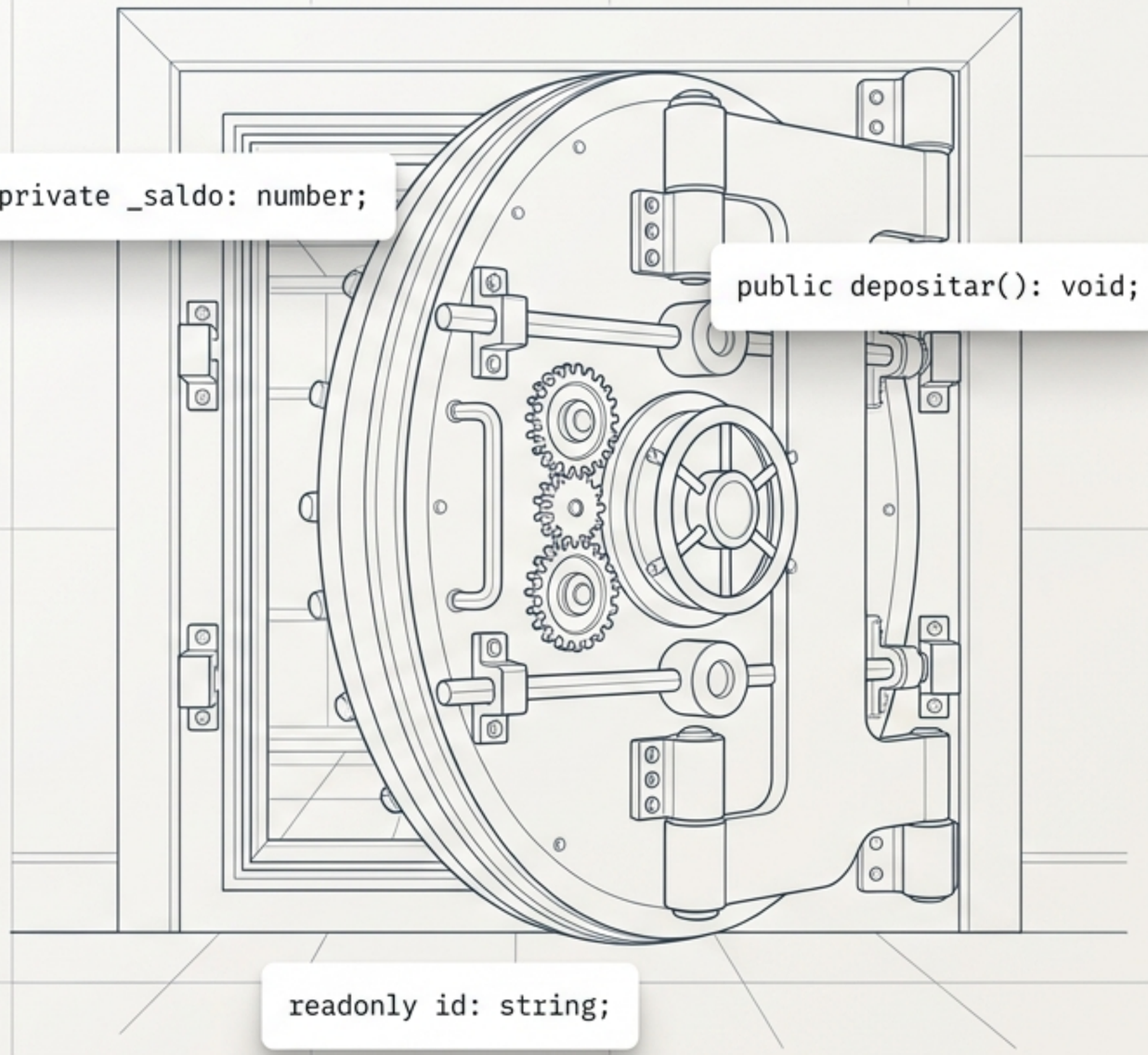
Aula 6: Encapsulamento e Modificadores de Acesso

Protegendo o estado interno e garantindo a integridade dos seus objetos em TypeScript.



Gemini Notebook

O Problema Motivador: Código Sem Fronteiras

```
class ContaBancaria {
  titular: string;
  saldo: number;

  constructor(t: string, s: number) {
    this.titular = t;
    this.saldo = s;
  }
}

const conta = new ContaBancaria('Ana', 1000);
conta.saldo = -5000;
conta.saldo = 0;
```

O que impede um programador de corromper este objeto?

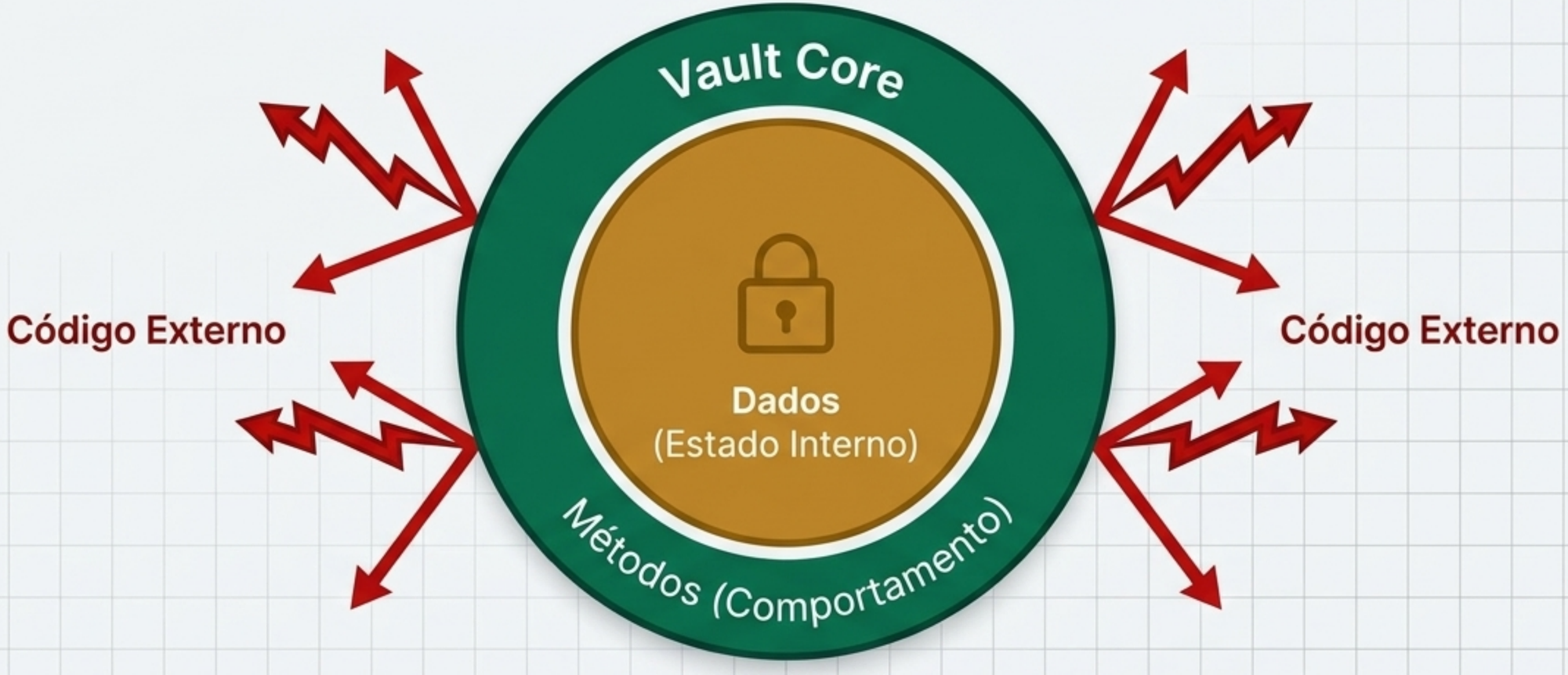
❌ Saldo negativo arbitrário sem validação lógica!

❌ Saldo zerado sem registro formal de saque!

Gemini Notebook

O Princípio do Encapsulamento

O objeto deve ser o único responsável por gerenciar suas regras de negócio.

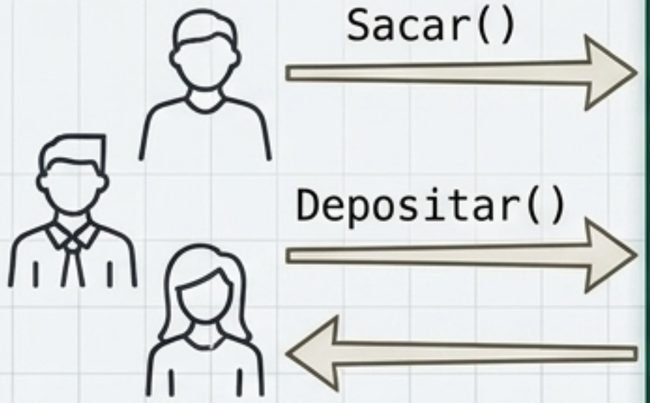


Mecanismo que vincula o código aos dados que ele trata, tornando-os **seguros** contra interferência externa.

Gemini Notebook

Interface Pública vs. Estado Privado

O Cliente (Código Externo)



Interface Pública

A única zona de contato seguro.
Métodos validados operam aqui.



Estado Privado (O Cofre)

```
saldo: number
```

A variável real vive aqui.
Intocável pelo lado de fora.

Gemini Notebook

Matriz de Visibilidade no TypeScript

	Mesma Classe	Subclasse (Herança)	Fora da Classe
public	✅	✅	✅
protected	✅	✅	❌
private	✅	❌	❌

Imutabilidade: readonly

Valor fixado na declaração ou no construtor. Não define quem vê, mas impede a modificação por qualquer ator do sistema.

Gemini Notebook

Exemplo Prático: Modificadores em Ação

```
class Pessoa {
  public nome: string;
  private cpf: string;
  readonly nascimento: Date;
}

const p = new Pessoa();
p.nome = 'Carlos'; // ✅
p.cpf = '000'; // ❌
p.nascimento = new Date(); // ❌
```



Acesso Livre em qualquer camada do sistema.



Trancado na Classe. Acessível APENAS pelo próprio objeto.



Apenas Leitura. Intocável após finalização do construtor.

Gemini Notebook

Imutabilidade Garantida: O Modificador Readonly

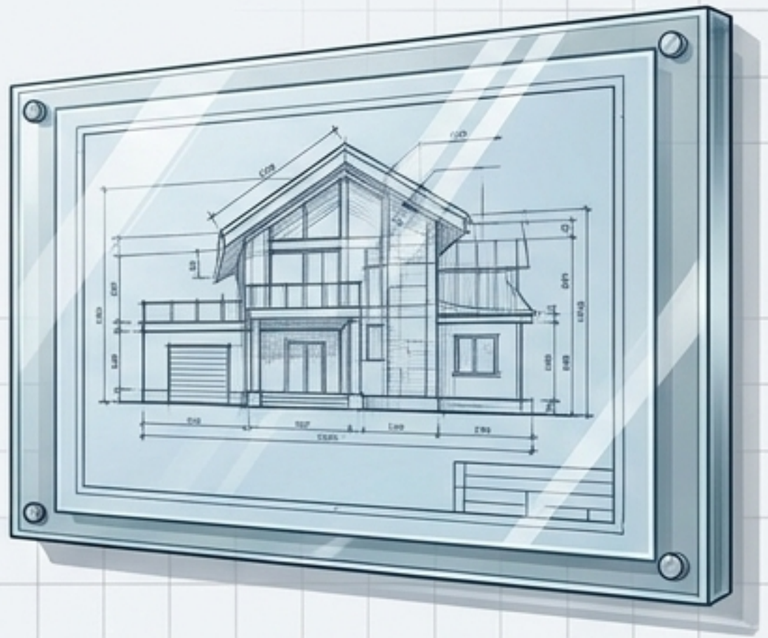
Ideal para configurações, IDs imutáveis e constantes de instância.

Fase 1: O Construtor



Atribuição Permitida (Apenas uma vez vez na declaração ou no corpo do construtor).

Fase 2: Execução



Estado Imutável Congelado. Qualquer tentativa de reatribuição gera erro no compilador TypeScript.

Gemini Notebook

O Açúcar Sintático: Parameter Properties

A Abordagem Verbosa

```
class Produto {
  public nome: string;
  private preco: number;

  constructor(nome: string, preco: number)
     this.nome = nome;
     this.preco = preco;
  }
}
```



A Abordagem Sênior

```
class Produto {
  constructor(
     public nome: string,
     private preco: number
  ) {}
}
```

Ao adicionar o modificador de acesso diretamente nos parâmetros do construtor, o TypeScript declara e atribui a propriedade automaticamente!

Gemini Notebook

Insight de Especialista: Compilação vs. Execução

TypeScript: private

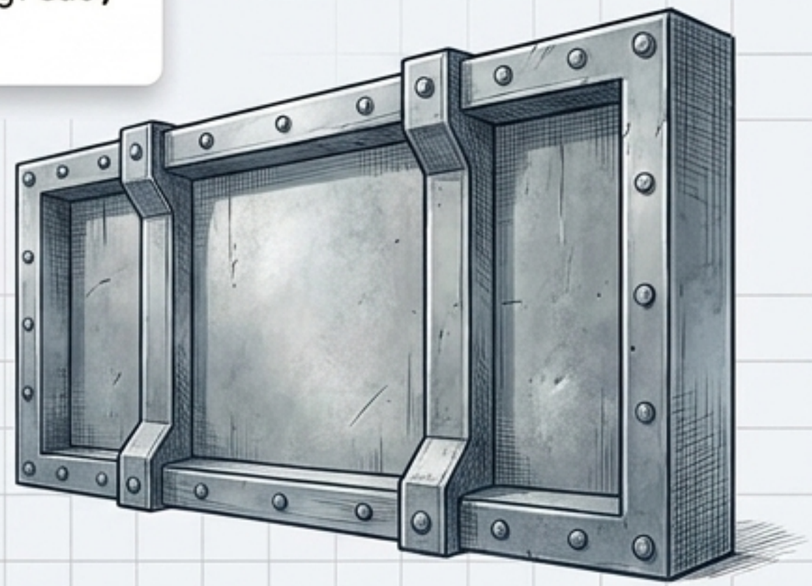
```
class TS {
  private segredo;
}
```



Ilusão de Compilação. No JavaScript final convertido, a variável vira uma propriedade normal, acessível e vulnerável em tempo de execução.

JavaScript Nativo:

```
class JS {
  #segredo;
}
```

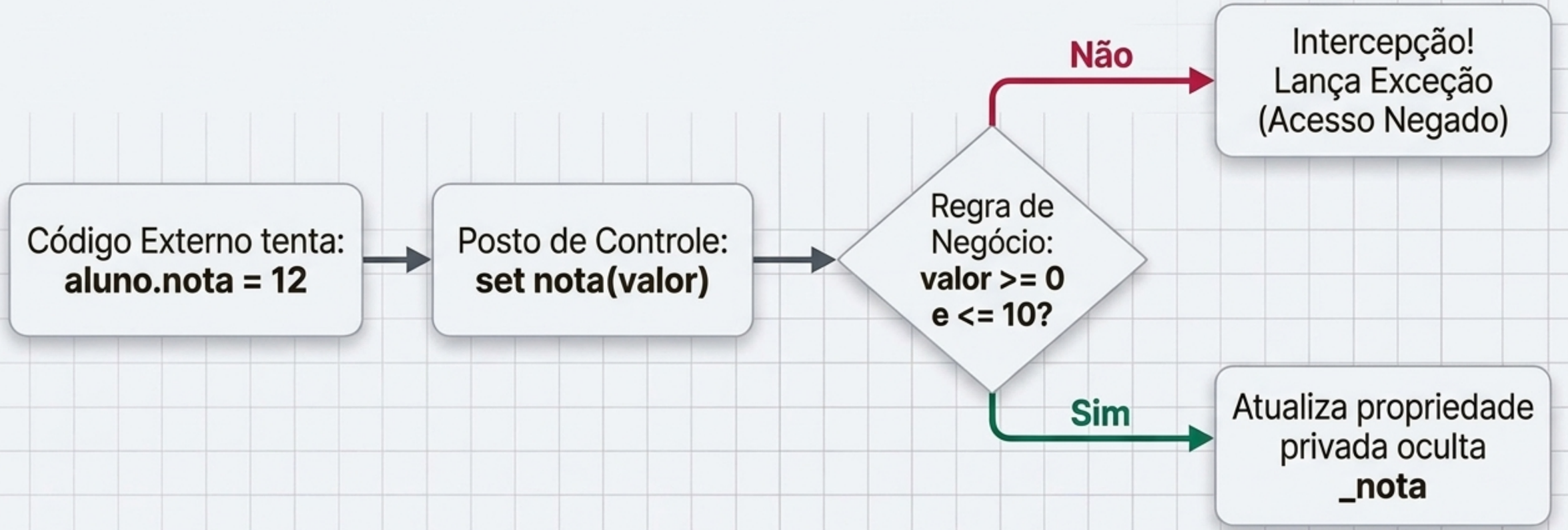


Segurança Real. A sintaxe ECMAScript 2022 garante um verdadeiro campo privado, inacessível externamente em tempo de execução.

Gemini Notebook

Métodos de Acesso: Assumindo o Controle (Get e Set)

Os postos de controle alfandegário do seu objeto.



Gemini Notebook

Implementando Getters e Setters

Convenção de Mercado:
O uso do underline (_) evita colisão de nomes entre a propriedade privada e o getter/setter público.

```
class Aluno {
  private _nota: number = 0;

  get nota(): number {
    return this._nota;
  }

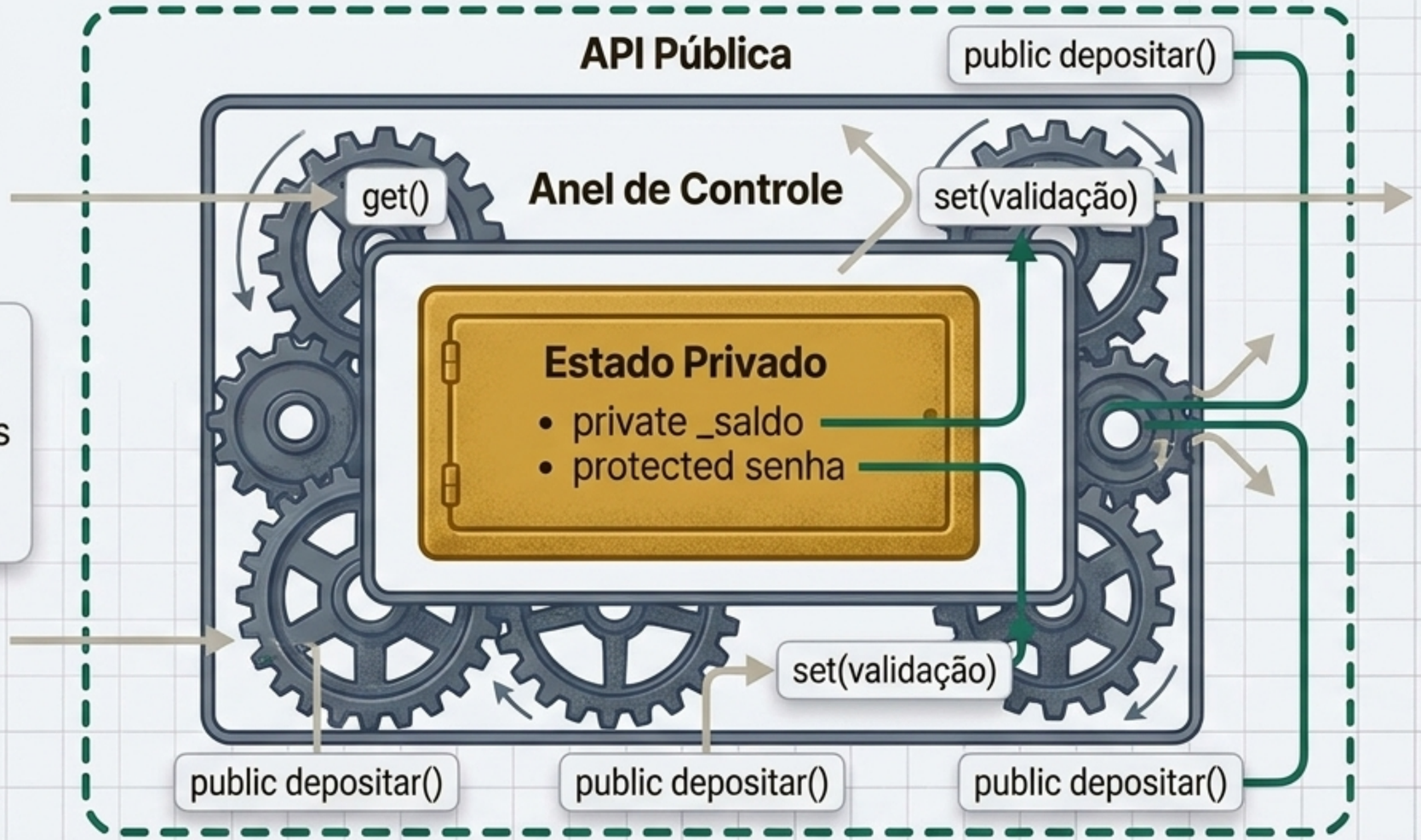
  set nota(valor: number) {
    if (valor < 0 || valor > 10) {
      throw new Error('Nota inválida!');
    }
    this._nota = valor;
  }
}
```

O Firewall:
Validação lógica bloqueia a entrada de dados corrompidos antes que toquem no estado interno.

Gemini Notebook

Síntese: A Anatomia do Objeto Blindado

O objeto perfeito isola seus dados e fornece ferramentas seguras para interação externa.



Gemini Notebook

Arquitetura Garantida: Resumo da Aula



Proteja Suas Regras: O objeto deve ser um motor autossuficiente e proteger rigorosamente suas próprias regras de negócio (Encapsulamento).



Tranque por Padrão: Use private e protected para ocultar estados internos delicados. Exponha apenas o estriitamente necessário.



Código Cirúrgico: Utilize Parameter Properties diretamente no construtor para criar classes enxutas, sem verbosidade e fáceis de ler.



Filtros de Entrada: Implemente Getters e Setters como firewalls lógicos para impedir a atribuição de dados falhos em tempo de execução.

Gemini Notebook